

When Idealized Travel Times Misrank Multi-Robot Schedules: Execution-Grounded Learning for Capacitated Pickup and Delivery

Anonymous Authors

Abstract—Learned dispatchers often evaluate robot transport with fixed travel times, although deployed robots share paths and exclusive stations. We study offline capacitated multi-robot pickup and delivery by applying an idealized decoder and a fixed collision-aware executor to identical complete schedules. On 100 instances with 60 jobs and 16 robots, a fixed 17-generator comparison set gives a mean per-instance Kendall τ_b of 0.334 ± 0.026 , a $33.0 \pm 1.3\%$ inversion rate, and $29.8 \pm 3.7\%$ selection regret. We then train a constructive graph policy with PPO from executor-evaluated terminal makespan while a calibrated event-based surrogate advances partial schedules and exposes projected station queues, committed workload, local robot density, and action-specific waiting. At 60 jobs the surrogate raises τ_b from 0.446 to 0.730, lowers selection regret from 4.1% to 1.2%, and scores a schedule about $290\times$ faster than the executor. A matched 2×2 factorial shows that executor-grounded training reduces single-rollout makespan by 15.9% relative to the fully observed idealized-objective arm; congestion observability adds a further 3.0%. With eight executor-scored policy rollouts, the three-seed policy improves makespan by 5.9% over the test-set best fixed dispatching rule and by 3.6% over a genetic baseline. The results separate inexpensive congestion projection during construction from the executor that defines solution value.

Index Terms—Intelligent and Flexible Manufacturing, Planning, Scheduling and Coordination, Reinforcement Learning, Multi-Robot Systems

I. INTRODUCTION

Fleets of autonomous mobile robots (AMRs) increasingly perform internal transfers in factories and terminals. In the fixed layout studied here (Fig. 1), jobs arriving at inbound stations must be consolidated on capacitated robots and delivered to outbound stations across a shared floor. The scheduler selects the carrier and order of each pickup and delivery. Its decisions are evaluated in an execution setting in which robots require collision-free trajectories and each station serves at most one robot at a time and provides only finite waiting space.

Learned constructive schedulers encode partial schedules with graph or attention models and have progressed from operation-machine decisions to transport-aware operation-machine-vehicle choices [1]–[5]. These methods commonly represent transport through fixed pairwise travel times. In our pickup-and-delivery setting, that abstraction does not reproduce delays caused by route reservations or non-preemptive contention at shared inbound and outbound stations.

Assignment and routing have also been integrated directly, through routed-cost task assignment and reinforcement-learning frameworks coupled with conflict-free path planners [6]–[8]. These studies establish the value of routing-aware assignment, but their principal questions concern online

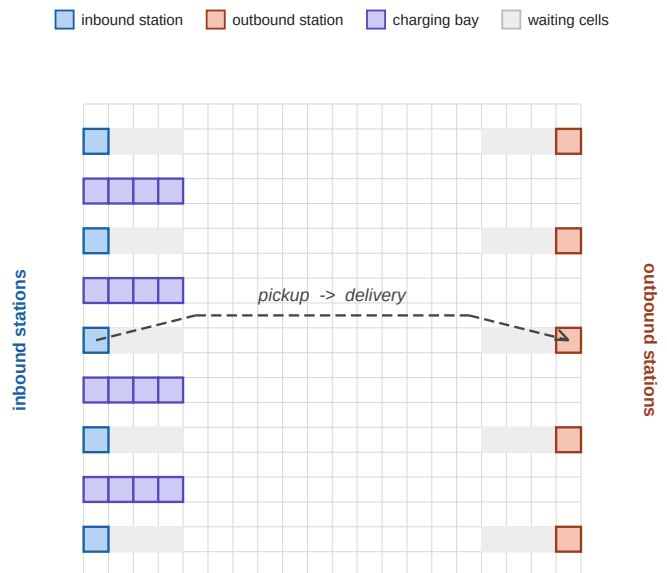


Fig. 1. Fixed 20×19 layout used throughout. Five inbound and five outbound stations have single-file, depth-three waiting lines. Sixteen dedicated charging bays occupy four aisle rows. The dashed arrow illustrates one pickup-delivery pair.

allocation or per-instance optimization rather than how route conflicts and contention at exclusive stations with finite waiting space affect evaluator fidelity over a common set of complete offline schedules. This paper addresses that complementary question by quantifying how an idealized travel-time model distorts both the predicted makespan and the ranking of complete schedules when robot capacity, non-zero station service times, finite waiting space, and collision-aware execution interact.

We study this question through a paired comparison. For the same assignment and operation sequence, an idealized decoder charges free-space travel and starts service on arrival, whereas the fixed collision-aware executor plans space-time paths and enforces exclusive service. By evaluating the same schedule under both models, the execution penalty isolates makespan-estimation error from differences in schedule construction. More importantly, we compare the orderings induced by the two evaluators: if they disagree, the idealized evaluator is not merely miscalibrated but can prefer a schedule that executes worse. The executor jointly captures route-reservation delays and contention at exclusive stations; we evaluate their combined effect on schedule value and ranking rather than attribute the gap to individual delay

sources.

The same distinction guides the learning method. We train a constructive policy with PPO using a terminal reward derived from makespan returned by the fixed collision-aware executor. During both training and inference, however, partial schedules are advanced by a calibrated event-based surrogate rather than by the executor. The policy therefore observes surrogate-projected station availability, queue occupancy, and outstanding committed workload, together with projected local robot density and an estimated wait for each candidate action. The surrogate’s affine travel-time correction and queue-depth-conditioned waiting penalties are fitted offline to operation-level traces generated by the fixed collision-aware executor. This calibration is designed to reduce systematic drift in the intermediate robot and station clocks while avoiding space–time routing at every construction step. Once a schedule is complete, the fixed executor evaluates its makespan; during training, this executor-evaluated cost supplies the terminal reward. Thus, execution grounding refers to the optimization target, whereas intermediate state construction remains surrogate-based.

On the 60-job test set, the idealized and executed rankings have mean $\tau_b = 0.334 \pm 0.026$ and invert $33.0 \pm 1.3\%$ of comparable pairs; choosing by the idealized evaluator incurs $29.8 \pm 3.7\%$ execution regret within the comparison set. Executor-grounded training is the dominant factor in the ablation, while the benefit of exposing projected congestion grows from the 40- to the 100-job setting. The three-seed policy with eight executor-scored rollouts reduces makespan by 5.9% relative to the test-set best fixed rule and by 3.6% relative to the genetic baseline at 60 jobs.

This study asks when an idealized travel-time model fails to preserve the ranking of complete schedules, and whether executor-grounded learning with surrogate-projected congestion features can reduce the resulting schedule-selection error. Specifically, this paper makes three contributions:

- A paired evaluation of complete offline capacitated AMR schedules under an idealized transport decoder and a fixed collision-aware executor. The evaluation quantifies value error through the execution penalty and ranking error through Kendall’s τ_b and the directly measured ranking inversion rate.
- A controlled workload-scale experiment that fixes the fleet and layout, varies the number of jobs, and maps the non-monotone regimes in which the two evaluators rank complete schedules differently. Value error, ranking inversion, selection regret, and routing-failure rate characterize the associated conditions.
- An executor-grounded constructive policy trained with PPO using a terminal reward derived from makespan returned by the fixed collision-aware executor. During both training and inference, a calibrated event-based surrogate advances partial schedules and supplies projected station states, local robot density, and action-specific waiting estimates. A 2×2 factorial evaluation crosses this surrogate-projected congestion representation with the source of the training objective at matched

computational budgets.

Section II reviews learned scheduling and integrated task and path planning. Section III defines the scheduling problem, the two evaluators, and the value and ranking measures. Section IV presents the calibrated surrogate, congestion-aware policy architecture, and PPO training procedure. Section V reports the schedule–ranking analysis, workload-scale study, policy evaluation, and ablations. Section VI concludes the paper.

II. RELATED WORK

A. Learned Scheduling with Transport Abstractions

L2D [1] introduced a graph-based constructive policy for job shops, using the change in a makespan lower bound as a dense reward. Flexible job-shop extensions use heterogeneous operation–machine graphs and composite actions [2], and subsequent work develops scalable graph-based schedulers [3]. Transport-aware formulations additionally model AGV movement and operation–machine–vehicle decisions using graph, transformer, or multi-agent architectures [4], [5], [9].

The cited transport-aware schedulers principally represent motion with fixed travel times between resources. This supports inexpensive partial-schedule evaluation but does not reproduce delays from shared-space reservations or exclusive pickup and delivery service. We therefore build on established graph-based constructive schedulers and focus on a different question: whether an idealized evaluator accurately predicts executed makespan and preserves schedule rankings under collision-aware execution.

B. Integrated Assignment, Routing, and Service Resources

Scheduling and material handling have long been recognized as coupled decisions [10], [11]. Conflict-aware methods integrate dispatch and route search for capacitated AGVs [12], container-terminal operations [13], and just-in-time delivery [14]. Integrated MAPD likewise uses routed delivery cost to inform assignment and includes a multi-load variant [6]. These methods optimize routing-aware solutions per instance rather than learning the offline constructive policy considered here.

Learning-based integration is also established. Li *et al.* [7] train a dynamic task-assignment policy whose action rewards are evaluated by a lower-level conflict-free path generator. Wang *et al.* [8] jointly train task-allocation and path-planning agents, while a reinforcement-learning hyper-heuristic combines task sequencing with conflict-avoidance rules but re-runs search for each instance [15].

Multi-agent path finding computes collision-free space–time trajectories on shared graphs [16]; lifelong MAPD combines repeated routing with online task service [17]. Learned components also appear inside the planner, for example as priority policies for rolling-horizon prioritized planning [18]. Commitment timing is also consequential: deliberate assignment deferral changes future reservations and deadline performance [19]. Nor are service resources absent

from the literature; recent multi-subsystem MAPD work explicitly coordinates single-dock handover stations with finite buffers [20]. These studies establish that assignment, routing, commitment, and service contention can be modeled jointly. The distinction here is the evaluation question: we apply an idealized decoder and a fixed deployment executor to a common set of complete offline schedules, making evaluator-induced value and ranking errors directly measurable.

C. Learning from Terminal Cost

Policy-gradient methods can optimize sequence-level objectives from a terminal return [21], and PPO stabilizes policy updates through a clipped surrogate objective [22]. In neural combinatorial optimization, rollout baselines reduce variance when solution cost is available only after construction is complete [23]. These methods address learning from delayed solution-level feedback, but they do not by themselves determine whether that feedback reflects deployment execution. In our method, executor grounding applies to the terminal cost rather than to intermediate transitions. Both training and inference advance partial schedules with a calibrated surrogate, while the fixed collision-aware executor evaluates completed schedules. During training, its makespan supplies the terminal cost; the surrogate-projected state does not define the objective.

Taken together, prior work establishes learned schedule construction, routing-aware assignment, and explicit treatment of service resources. The cited literature does not make evaluator fidelity over a common set of complete offline schedules its primary object of study. We quantify both value and ranking errors induced by an idealized transport model and train PPO against executor-evaluated terminal cost while exposing surrogate-projected congestion information during schedule construction.

III. PROBLEM FORMULATION

A. System, Jobs, and Schedules

Table I provides a consolidated reference for the notation used in this section. The workspace is the obstacle-free, four-connected grid $\mathcal{W} = \{0, \dots, 19\} \times \{1, \dots, 19\}$ with unit-time moves and Manhattan distance Δ . Five inbound stations and five outbound stations occupy the left and right boundaries at rows $\mathcal{Y} = \{2, 6, 10, 14, 18\}$:

$$\mathcal{D}^{\text{in}} = \{(0, y) : y \in \mathcal{Y}\}, \quad \mathcal{D}^{\text{out}} = \{(19, y) : y \in \mathcal{Y}\}. \quad (1)$$

Let $\mathcal{D} = \mathcal{D}^{\text{in}} \cup \mathcal{D}^{\text{out}}$. Each station is an exclusive, non-preemptive server. For a station d , let $\mathbf{u}_d$ be the inward wall normal. Its finite single-file waiting set is

$$W_d = \{d + r\mathbf{u}_d \in \mathcal{W} \setminus \mathcal{D} : r \in \{1, \dots, L\}\}, \quad L = 3. \quad (2)$$

Thus, each queue extends three cells inward from its station and leaves the two lateral departure cells free; each waiting cell holds at most one AMR. The fleet is fixed at $m = 16$. Its dedicated bays occupy four aisle rows between station rows,

$$\mathcal{B} = \{0, 1, 2, 3\} \times \{4, 8, 12, 16\}, \quad (3)$$

with a one-to-one assignment $b_k \in \mathcal{B}$ for each AMR $k \in \mathcal{M}$. An idle AMR occupies its bay, and every AMR returns to its bay after completing its assigned work. Figure 1 shows this fixed geometry.

An instance contains jobs $\mathcal{J} = \{1, \dots, n\}$, where each job represents one parcel transfer. Job j has size $\sigma_j \in \mathcal{S} = \{A, B, C\}$, with $A \prec B \prec C$, origin $o_j \in \mathcal{D}^{\text{in}}$, destination $g_j \in \mathcal{D}^{\text{out}}$, and deterministic service time

$$p_j = p(\sigma_j), \quad p(A) = 5, \quad p(B) = 10, \quad p(C) = 15. \quad (4)$$

The same duration is required for pickup ρ_j and delivery δ_j , and all jobs are available at time zero. Pickup and delivery must be performed by the same AMR; transshipment is not allowed.

Each AMR has one rack position of each class. A smaller job may occupy a larger position, but not conversely. With $N_s^k(t)$ denoting the number of onboard class- s jobs and $Q_s = 1$ for every $s \in \mathcal{S}$, the nested capacity constraints are

$$\sum_{s \succeq \varsigma} N_s^k(t) \leq \sum_{s \succeq \varsigma} Q_s, \quad k \in \mathcal{M}, \varsigma \in \mathcal{S}, t \geq 0. \quad (5)$$

Equivalently, $N_C^k \leq 1$, $N_B^k + N_C^k \leq 2$, and $N_A^k + N_B^k + N_C^k \leq 3$.

A complete schedule is $x = (\mathbf{a}, \pi)$. The vector $\mathbf{a} \in \mathcal{M}^n$ assigns each job to one AMR, while π is a priority list containing the $2n$ pickup and delivery events. It obeys $\rho_j \prec_\pi \delta_j$, and every prefix of each robot's induced subsequence satisfies (5). Let $\mathcal{X}$ denote the set of schedules satisfying these assignment, precedence, and rack constraints. The list π fixes event priority, not physical start times: station service, waiting, and routed movement are supplied by an evaluator.

B. Paired Schedule Evaluators

We apply two deterministic evaluators to the same $x \in \mathcal{X}$. For robot k , let n_k be its number of events and $\pi^k = (u_1^k, \dots, u_{n_k}^k)$ the subsequence induced by x . Let $\ell(u)$ and $j(u)$ be an event's station and job, respectively, and set $\ell(u_0^k) = b_k$. The idealized evaluator $\tilde{\Phi}$ assumes independent robots, immediate service on arrival, and free-space travel:

$$\begin{aligned} \tilde{C}_k(x) &= \sum_{i=1}^{n_k} \left[\Delta(\ell(u_{i-1}^k), \ell(u_i^k)) + p_{j(u_i^k)} \right] \\ &\quad + \Delta(\ell(u_{n_k}^k), b_k), \quad \tilde{C}(x) = \max_{k \in \mathcal{M}} \tilde{C}_k(x). \end{aligned} \quad (6)$$

It therefore ignores inter-robot conflicts, station exclusivity, finite waiting space, and delays caused by reservations. Global interleavings with the same per-robot subsequences may consequently tie under $\tilde{\Phi}$.

The deployment evaluator Φ is a fixed prioritized executor. It processes events in π , routes each leg with space-time A* over a shared reservation table, and admits unit moves or waits. A move is rejected if its next cell is reserved at the current or next tick; this prevents vertex conflicts, head-on exchanges, and immediate following into a vacated cell. Each station serves one AMR at a time. If service cannot start on

TABLE I
NOTATION.

Symbol	Meaning and value/domain	Symbol	Meaning and value/domain
<i>System, jobs, and schedules</i>		<i>Evaluation and error measures</i>	
$d, k, j, s, \varsigma, t, i$	station, AMR, job, size-class, class-threshold, time/epoch, and event/instance indices	$\tilde{\Phi}, \Phi$	idealized evaluator and collision-aware deployment executor
$\mathcal{W}$	four-connected workspace, $\{0, \dots, 19\} \times \{1, \dots, 19\}$	π^k, u_i^k, n_k	event subsequence of AMR k , its i th event, and number of events
$\Delta(u, v)$	Manhattan distance between cells u and v	$\ell(u), j(u)$	station and job associated with event u
$\mathcal{Y}$	station rows, $\{2, 6, 10, 14, 18\}$	$\tilde{C}_k, \tilde{C}$	idealized completion time of AMR k and idealized makespan
$\mathcal{D}^{\text{in}}, \mathcal{D}^{\text{out}}, \mathcal{D}$	inbound stations, outbound stations, and their union	H	space-time A* search horizon, 100 ticks
$\mathbf{u}_d, W_d, L$	inward wall normal, waiting cells of station d , and queue length $L = 3$	$\nu_{\Phi}(x)$	routing or operation events that Φ cannot decode
$\mathcal{M}, m$	AMR set and fleet size, $m = 16$	$\mathcal{F}_{\Phi}$	schedules routable by the fixed executor, i.e., $\nu_{\Phi}(x) = 0$
$\mathcal{B}, b_k$	set of dedicated bays and the bay assigned to AMR k	$F_k^{\Phi}, C_{\max}^{\Phi}$	executed completion time of AMR k and executed makespan
$\mathcal{J}, n$	job set and number of jobs	x^*	executor-relative makespan-minimizing schedule
$\mathcal{S}, \sigma_j$	size classes $\{A, B, C\}$, $A \prec B \prec C$, and class of job j	$\Lambda(x)$	relative value error (execution penalty) of $\tilde{\Phi}$
o_j, g_j	inbound origin and outbound destination of job j	$\mathcal{C}_i$	common executor-routable candidate set for test instance i
$p_j = p(\sigma_j)$	pickup/delivery service time; $p(A), p(B), p(C) = 5, 10, 15$	P_i, D_i	concordant and discordant schedule-pair counts
ρ_j, δ_j	pickup and delivery events of job j	$T_i^{\tilde{\Phi}}, T_i^{\Phi}$	pair counts tied only by the indicated evaluator
$Q_s, N_s^k(t)$	class- s rack positions ($Q_s = 1$) and onboard class- s jobs	$\tau_{b,i}, \text{Inv}_i$	tie-corrected Kendall coefficient and inversion rate
$x = (\mathbf{a}, \pi)$	complete schedule: assignment vector and global event-priority list	$\tilde{x}_i, x_i^{\Phi}, \text{Reg}_i$	selections of the two evaluators and executor-relative selection regret
$\mathbf{a}, \pi$	job-to-AMR assignments and a precedence-feasible list of $2n$ events		
$\mathcal{X}$	schedules satisfying assignment, precedence, and rack constraints		
<i>Constructive decision process</i>			
s_t	state at construction epoch t		
$\mathbf{X}_t^A, \mathbf{X}_t^J$	projected AMR and job states		
$\mathbf{E}_t, \pi_{1:t}$	committed station-service windows and current event-list prefix		
$\mathbf{a}_t = (\omega, k, j)$	composite action selecting operation, AMR, and job		
ω	operation type, PICK or DEL		
$\hat{\Psi}$	calibrated event-based surrogate transition		

arrival, the executor reserves an available cell in W_d ; when none is available, the AMR remains at its upstream location until the next relevant release time. After its last event, each AMR is routed back to b_k .

The space-time search horizon is $H = 100$ ticks. Let $\nu_{\Phi}(x)$ count routing or operation events that the decoder cannot complete within its rules, and define the executor-relative routable set

$$\mathcal{F}_{\Phi} = \{x \in \mathcal{X} : \nu_{\Phi}(x) = 0\}. \quad (7)$$

For $x \in \mathcal{F}_{\Phi}$, let $F_k^{\Phi}(x)$ include all movement, waiting, service, and the return to b_k . The executed makespan and optimization objective are

$$C_{\max}^{\Phi}(x) = \max_{k \in \mathcal{M}} F_k^{\Phi}(x), \quad x^* \in \arg \min_{x \in \mathcal{F}_{\Phi}} C_{\max}^{\Phi}(x). \quad (8)$$

Because Φ is a bounded prioritized planner, $\mathcal{F}_{\Phi}$ is executor-relative rather than a claim of general MAPF feasibility. A failed decode contains a fixed horizon penalty rather than elapsed execution time. We therefore report routability separately and compute all execution-time and ranking statistics below only on schedules in $\mathcal{F}_{\Phi}$.

C. Value and Ranking Error

For the same executor-routable schedule, the relative value error of the idealized evaluator is the execution penalty

$$\Lambda(x) = \frac{C_{\max}^{\Phi}(x) - \tilde{C}(x)}{\tilde{C}(x)}. \quad (9)$$

Value error alone does not show whether the abstraction changes a scheduling decision. For test instance i , let $\mathcal{C}_i \subseteq \mathcal{F}_{\Phi}$ be the common set of complete schedules produced by all evaluated methods. For every unordered pair $\{x, y\} \subset \mathcal{C}_i$, define it as concordant when

$$[\tilde{C}(x) - \tilde{C}(y)][C_{\max}^{\Phi}(x) - C_{\max}^{\Phi}(y)] > 0, \quad (10)$$

and discordant when the product is negative. Let P_i and D_i be the respective counts, and let $T_i^{\tilde{\Phi}}$ and T_i^{Φ} count pairs tied only by the indicated evaluator. Kendall's tie-corrected coefficient is

$$\tau_{b,i} = \frac{P_i - D_i}{\sqrt{(P_i + D_i + T_i^{\tilde{\Phi}})(P_i + D_i + T_i^{\Phi})}}. \quad (11)$$

The directly measured ranking-inversion rate is

$$\text{Inv}_i = \frac{D_i}{P_i + D_i}, \quad (12)$$

so pairs tied by either evaluator are excluded from its denominator and are reported separately. Finally, using a

predetermined tie-breaking rule, let $\tilde{x}_i = \arg \min_{x \in \mathcal{C}_i} \tilde{C}(x)$ and $x_i^\Phi = \arg \min_{x \in \mathcal{C}_i} C_{\max}^\Phi(x)$. Their executor-relative selection regret is

$$\text{Reg}_i = \frac{C_{\max}^\Phi(\tilde{x}_i) - C_{\max}^\Phi(x_i^\Phi)}{C_{\max}^\Phi(x_i^\Phi)}. \quad (13)$$

The three ranking measures are computed per instance before aggregation, so instances with more sampled schedules do not receive unintended extra weight.

D. Constructive Decision Process

The policy constructs one schedule in $2n$ epochs. At epoch t , its state and composite action are

$$s_t = (\mathbf{X}_t^A, \mathbf{X}_t^J, \mathbf{E}_t, \pi_{1:t}), \quad (14)$$

$$a_t = (\omega, k, j), \quad \omega \in \{\text{PICK}, \text{DEL}\}. \quad (15)$$

Here $\mathbf{X}_t^A$ and $\mathbf{X}_t^J$ describe the projected robot and job states, $\mathbf{E}_t$ stores committed station service windows, and $\pi_{1:t}$ is the current prefix. The action mask allows pickup of an unpicked job only by an AMR with compatible rack capacity, and allows delivery only after pickup by that job's carrier. It therefore enforces assignment, precedence, and nested capacity during construction.

Both training and inference advance a prefix with the calibrated event-based surrogate

$$s_{t+1} = \hat{\Psi}(s_t, a_t), \quad (16)$$

which supplies inexpensive projected travel, station, and waiting states. It does not determine the optimization target. Once a complete schedule has been constructed, the fixed executor Φ evaluates it; PPO receives the negative executor-evaluated makespan as its terminal return on routable rollouts. Failed rollouts receive the executor's finite training-only score, while their returned duration is excluded from reported makespan statistics. Section IV specifies $\hat{\Psi}$, the observable congestion features, and the policy parameterization.

E. Scope and Assumptions

Robots are homogeneous, all jobs are known before construction, and service times are deterministic and class-dependent. Here *offline* describes the scheduling instance, not offline reinforcement learning. The controlled study fixes the layout, the $m = 16$ fleet, and the executor, while varying workload through n . Battery consumption, turning and acceleration costs, localization error, transshipment, and on-line arrivals are excluded. Consequently, the results identify workload-dependent evaluator error within this controlled system; generalization to other layouts, fleets, and routing executors remains outside the present scope.

IV. EXECUTION-GROUNDED CONSTRUCTIVE POLICY

At each of the $2n$ decision epochs, the policy commits one composite action $a_t = (\omega, k, j)$ and never revises it. Its EXTENDGNN encoder gives stations their own node type because station occupancy, unlike a fixed travel matrix, depends on earlier commitments. Station nodes carry projected service windows $\mathbf{E}_t$ and exchange information with robots before jobs are encoded.

TABLE II
DECISION-STATE FEATURES.

Group	Channels
Robot $\mathbf{x}_k^A$ (12)	busy flag and residual busy time $\tau_k - t_{\text{now}}$; onboard counts per class and free nested headroom; jobs currently carried; adjacency to a station or waiting cell; distance to the nearest station; local robot density within Manhattan radius 2; position
Station $\mathbf{x}_d^D$ (9)	inbound/outbound indicator; availability delay $(\alpha_d - t_{\text{now}})^+$; committed queue depth, both per robot and relative to $ W_d $ and clipped at one; residual service time of the window in progress; total committed workload still outstanding; position
Job $\mathbf{x}_j^J$ (15)	service time p_j ; time since release; carrier index and status; size class σ_j one-hot; origin and destination indices; the best-case completion time of j over all robots under $\hat{\Psi}$; origin and destination positions
Action $\phi_t(\omega, k, j)$ (3)	surrogate travel from x_k to the target station; the downstream pickup-to-delivery distance, zero for a delivery; the queueing wait $\hat{\Psi}$ predicts at that station on arrival

A. Decision State

Between commitments the state is advanced by the calibrated surrogate $\hat{\Psi}$ of Section IV-B, not by the executor Φ : Φ prices a complete schedule, and running it once per epoch is neither affordable nor well defined on a prefix. After the prefix $\pi_{1:t}$, $\hat{\Psi}$ maintains for each robot k a position x_k , a ready time τ_k , and a class-count vector N^k ; for each station d an availability α_d and the list of windows $E_d^t = \{(\underline{s}_e, \bar{s}_e)\}$ it has already committed; and for each job j its status (unassigned, carried, delivered) and carrier. The decision clock is $t_{\text{now}} = \min_k \tau_k$. This bookkeeping is the concrete form of the tuple $(\mathbf{X}_t^A, \mathbf{X}_t^J, \mathbf{E}_t, \pi_{1:t})$ in (14); the station features $\mathbf{X}_t^D$ below are a readout of $\mathbf{E}_t$ at t_{now} .

Table II lists the four feature groups. All channels are normalised to $[0, 1]$ by quantities resolved from the live layout—floor extent, the binding suffix cap $\sum_{s \succeq \varsigma} Q_s$ of (5), and fleet size. These scales remain fixed across the workload study, so changing the number of jobs changes the scheduling load rather than the numerical range of the feature channels.

B. Calibrated Event Surrogate

The surrogate advances only event clocks; it neither reserves grid cells nor plans paths. For Manhattan distance $d > 0$, its calibrated leg time is

$$\hat{t}(d) = \max\{d, 0.891719d + 4.492501\}. \quad (17)$$

If a robot is ready at station q at time r , its predicted wait is

$$\begin{aligned} \hat{w}(q, r) &= (\alpha_q - r)^+ + \beta_{\min\{z(q, r), 3\}}, \\ \beta &= (0.317, 0.317, 2.236, 6.415), \end{aligned} \quad (18)$$

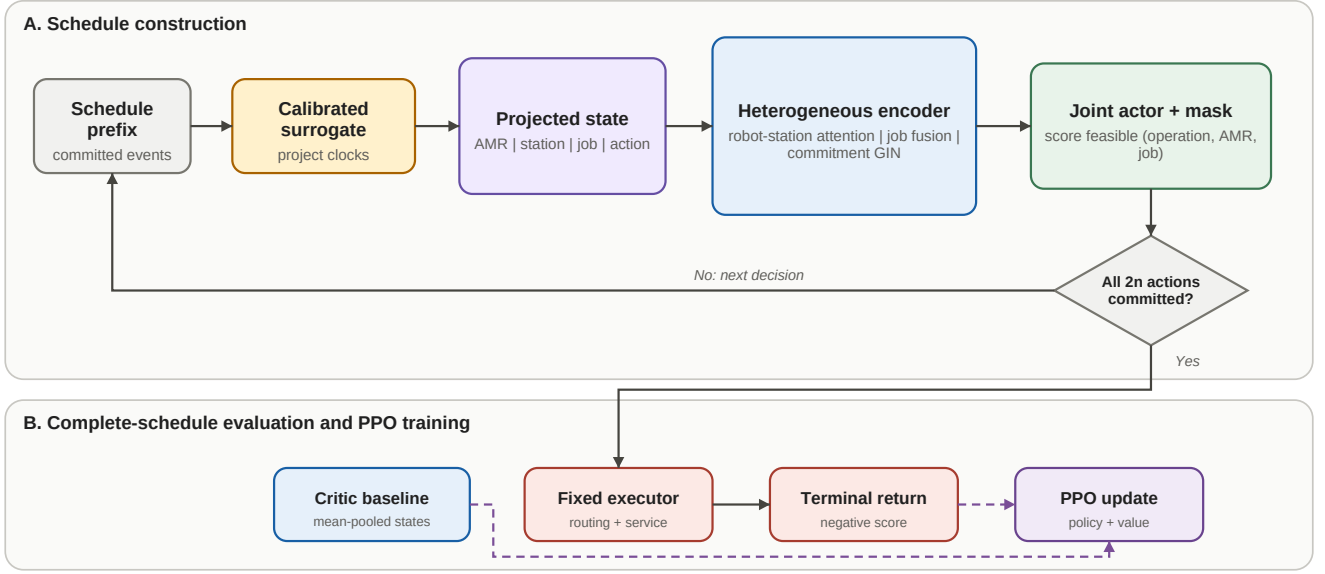


Fig. 2. Overview of the execution-grounded constructive policy. After each feasible operation–AMR–job decision, the completion check either returns the updated prefix for another decision or passes the complete schedule to the fixed executor. The terminal score and critic baseline provide the dashed PPO training inputs.

where $z(q, r)$ counts previously committed service windows that end after r . The action arrives after (17), starts after (18), commits a non-preemptive interval of length p_j , and updates the selected robot and station clocks. Thus, projected queue depth and workload reflect the schedule prefix, but not the executor’s eventual space–time reservations.

We fitted (17)–(18) offline on 159 routable schedules from independently generated instances; one failed decode was discarded. The corpus supplied 10,280 routed legs. Calibration reduced travel-time MAE from 2.575 to 2.179 ticks and removed the mean per-leg bias (affine $R^2 = 0.875$). Queue penalties are bin means of extra wait beyond station availability, constrained to be nondecreasing with depth. This small model is used to reduce prefix-construction cost, not as evidence that the executor has been reproduced; Section V-B measures the remaining fidelity gap directly.

C. Entity Encoders

Each group is embedded into $\mathbb{R}^h$, $h = 128$:

$$\mathbf{h}_k^A = \text{MLP}_A(\mathbf{x}_k^A), \quad \mathbf{h}_d^D = \text{MLP}_D(\mathbf{x}_d^D), \quad \mathbf{h}_j^J = \mathbf{W}_J \mathbf{x}_j^J, \quad (19)$$

collected as $\mathbf{H}^A \in \mathbb{R}^{m \times h}$, $\mathbf{H}^D \in \mathbb{R}^{|\mathcal{D}| \times h}$, and $\mathbf{H}^J \in \mathbb{R}^{n \times h}$, where $\mathcal{D} = \mathcal{D}^{\text{in}} \cup \mathcal{D}^{\text{out}}$.

D. Robot–Station Interaction

Write $\text{At}(\mathbf{Q}, \mathbf{K}) = \text{MHA}(\text{LN}(\mathbf{Q}), \text{LN}(\mathbf{K}), \text{LN}(\mathbf{K}))$ for pre-normalised multi-head attention with four heads. One round couples the two resource types,

$$\begin{aligned} \mathbf{H}^D &\leftarrow \mathbf{H}^D + \text{At}(\mathbf{H}^D, \mathbf{H}^A), \\ \mathbf{H}^A &\leftarrow \mathbf{H}^A + \text{At}(\mathbf{H}^A, \mathbf{H}^A), \\ \mathbf{H}^A &\leftarrow \mathbf{H}^A + \text{At}(\mathbf{H}^A, \mathbf{H}^D), \end{aligned} \quad (20)$$

in that order. Stations first read the approaching fleet, robots exchange load information, and then read the updated stations. Zero-initialized output projections make this residual block an identity at the start of training.

E. Job Encoding over the Commitment Graph

Each job fuses the embeddings of the two stations it must visit,

$$\mathbf{z}_j^{(0)} = \mathbf{h}_j^J + \text{MLP}_F([\mathbf{h}_j^J, \mathbf{h}_{o_j}^D, \mathbf{h}_{d_j}^D]). \quad (21)$$

The fusion is residual rather than replacing because $|\mathcal{D}| = 10$ station embeddings are shared among n jobs: a replacing fusion writes what jobs have in common over what distinguishes them.

The prefix induces an undirected graph $\mathbf{A}_t$ on jobs. Two jobs are adjacent if they are consecutive on the same robot, or consecutive at the same outbound station, in $\pi_{1:t}$; the graph is therefore empty at $t = 0$ and accumulates the commitments actually made. Three residual layers of graph isomorphism convolution refine the embeddings,

$$\begin{aligned} \mathbf{m}_j^{(\ell)} &= (1 + \epsilon_\ell) \mathbf{z}_j^{(\ell-1)} + \frac{1}{|\mathcal{N}_t(j)|} \sum_{i \in \mathcal{N}_t(j)} \mathbf{z}_i^{(\ell-1)}, \\ \mathbf{z}_j^{(\ell)} &= \mathbf{z}_j^{(\ell-1)} + \text{LN}(\text{MLP}_\ell(\mathbf{m}_j^{(\ell)})), \end{aligned} \quad (22)$$

with ϵ_ℓ learned and $\mathbf{z}_j \equiv \mathbf{z}_j^{(3)}$. Mean rather than sum aggregation keeps the message scale stable as $\mathbf{A}_t$ densifies over an episode; the neighbour term is zero when $\mathcal{N}_t(j)$ is empty.

Only committed sequence arcs enter $\mathbf{A}_t$; origin and destination relationships already enter through the shared station embeddings in (21).

F. Composite Actor Head

The action is scored jointly rather than factored into a job choice followed by a robot choice. Let $d(\omega, j) = o_j$ for $\omega = \text{PICK}$ and $d(\omega, j) = g_j$ for $\omega = \text{DEL}$, and let e_ω be a learned operation embedding. Each candidate triple receives the logit

$$\ell_t(\omega, k, j) = \text{MLP}_\pi([e_\omega, \mathbf{h}_k^A, \mathbf{z}_j, \mathbf{h}_{d(\omega, j)}^D, \phi_t(\omega, k, j)]), \quad (23)$$

an input of width $4h + 3$. The three surrogate channels ϕ_t enter here, at the only place where a robot, a job, and a station are named together; no pooled embedding can recover the travel and waiting cost of a specific pairing. Writing $\mathcal{A}_t$ for the legal set of Section III-D—pickup only by a robot whose nested rack admits σ_j , delivery only by the carrier of j —illegal triples are set to $-\infty$ and

$$\pi_\theta(a_t | s_t) = \frac{\exp \ell_t(a_t)}{\sum_{a \in \mathcal{A}_t} \exp \ell_t(a)}. \quad (24)$$

Masking guarantees assignment, precedence, and rack feasibility by construction; remaining decoder failures are recorded by ν_Φ in (7).

G. Critic

A single value head reads mean-pooled encodings, the job pool taken over undelivered jobs only:

$$V_\psi(s_t) = \text{MLP}_V([\bar{\mathbf{z}}_t, \bar{\mathbf{h}}^A, \bar{\mathbf{h}}^D]). \quad (25)$$

The actor resolves individual entities, whereas the critic supplies a compact baseline through three pooled vectors. Its loss also trains the shared encoder.

H. PPO Training

A sampled action sequence is decoded to a complete schedule x . Let $S_\Phi(x)$ equal $C_{\max}^\Phi(x)$ for a routable schedule and the executor’s finite training score otherwise. In the executor-grounded arms, the undiscounted terminal return is

$$R(x) = -S_\Phi(x), \quad (26)$$

while reported makespan excludes failed decodes as specified in Section III-B. Because reward is terminal, every prefix has return-to-go $R(x)$, while the critic gives step-dependent advantages $\hat{A}_t = R(x) - V_\psi(s_t)$. PPO maximizes

$$\mathcal{L}^{\text{clip}} = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (27)$$

$$r_t = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t).$$

with a squared value loss. Returns and advantages are normalized over each batch. Crucially, $\hat{\Psi}$ still generates every intermediate s_t in both training and inference; only the complete-schedule cost in (26) comes from Φ . The ideal-objective ablation replaces this cost by $\tilde{C}(x)$ without changing transitions, architecture, or action mask.

I. Cost, Size, and Congestion Channels

The encoder costs $O((m + |\mathcal{D}|)^2 h + n^2 h)$ per epoch and the joint head scores at most $2mn$ candidates. The $h = 128$ network has 617,605 parameters, including 49,409 in the value head.

Congestion enters the policy through an identifiable set of channels: the local robot density in x^A , the five occupancy channels of x^D (availability delay, the two queue-depth measures, residual service, and committed workload), and the predicted wait in ϕ_t . Everything else the network sees is geometry, capacity, or workload that an idealized transport model would also supply. This grouping is what the observability factor of Section V manipulates, and it is why the factor can be varied without changing capacity: the architecture, its parameter count, and its action space are identical across the levels.

V. EXPERIMENTS

A. Protocol and Comparison Methods

We keep the 20×19 layout, the homogeneous $m = 16$ fleet, the station configuration, and the fixed executor Φ unchanged, and vary workload through $n \in \{20, 40, 60, 80, 100\}$. Job class, inbound station, and outbound station are sampled uniformly, and all jobs are released at zero. Training uses 5,000 independently generated 60-job instances. Fixed validation and test sets are mutually disjoint and are also disjoint from training; the test set contains 100 instances at each workload. Before test evaluation, every learned policy is selected by the same executor-based validation score, $C_{\max}^\Phi + 1000\nu_\Phi$, so failed decodes cannot be preferred to completed schedules.

The dispatching-rule library crosses ten job priorities with six AMR rules, giving 60 deterministic combinations. The job rules cover FIFO, short- and long-processing-time priorities, distance and station workload, predicted completion, material matching, milk-run batching, and random choice; the AMR rules cover availability, distance, predicted completion, load, material matching, and random choice. For the baseline comparison, one combination is selected on the validation split separately for each n and frozen before test evaluation. The genetic algorithm (GA) uses a population of 200 for 150 generations followed by collision-aware local improvement.

The learned-policy experiment crosses two observations with three terminal returns. The core observation O_{core} removes the projected congestion channels of Section IV-I, whereas O_{cong} retains them. Completed rollouts are priced by the raw idealized makespan $\tilde{C}$, the executor makespan $C_{\max}^\Phi$, or the calibrated surrogate makespan $\hat{\Psi}$. Configuration names concatenate the return and observation; Surr–Cong is the proposed configuration. All six arms use the same architecture, data, and 4,000-epoch PPO budget.

All arms use the same executor-based model-selection rule, preventing evaluator-specific checkpoint selection.

At inference we report one deterministic greedy rollout and best-of-eight sampling, which selects the routable can-

didate with the lowest executed makespan. Primary comparisons average three training seeds within each instance. The experimental unit is the instance, method comparisons are paired, and 95% intervals are $1.96s/\sqrt{N}$. Failed decodes receive the finite score above during training and model selection and are reported as failures. Because that score is not an elapsed completion time, makespan averages include completed schedules only.

B. Evaluator Fidelity

We first test whether $\tilde{\Phi}$ preserves schedule order rather than only absolute value. This broad comparison uses 19 candidates per instance spanning rules, GA, and learned policies; more than 99.6% are executor-routable. Table III excludes failed schedules and uses the metrics of Section III-B. Here n is the number of jobs and τ_b is the per-instance, tie-corrected Kendall coefficient with respect to Φ .

TABLE III
IDEALIZED-EXECUTOR RANKING.

n	τ_b	Inv. (%)	Reg. (%)
20	0.26 ± 0.04	36.25 ± 1.93	16.08 ± 2.73
40	0.13 ± 0.03	43.22 ± 1.54	38.62 ± 3.43
60	0.23 ± 0.03	38.40 ± 1.27	30.63 ± 3.79
80	0.30 ± 0.03	34.96 ± 1.30	27.51 ± 7.20
100	0.37 ± 0.02	31.39 ± 1.20	9.41 ± 4.04

The abstraction misranks schedules at every workload. At the 60-job training size, 38.4% of comparable pairs are inverted and selecting its minimum incurs 30.6% executed regret. Although ranking partially recovers at 100 jobs, $\tau_b = 0.37$ and 9.4% regret show that value calibration alone does not guarantee decision fidelity.

We next compare $\tilde{C}$ and $\hat{\Psi}$ with Φ on the same 19-candidate field: 12 competitive rules, all six factorial policy cells, and GA. The rules combine FIFO, SPT, LPT, milk-run, material-matching, or earliest-completion job priority with the earliest-available or earliest-completion AMR rule. They occupy mean ranks 3–30 in the full 60-rule benchmark; excluding clearly dominated random, nearest-station, and least-congested rules prevents mechanical inflation of rank agreement. The policies and GA also prevent $\hat{\Psi}$ from being tested only on schedules constructed by rules that already use it. Table IV reports within-instance decision error and the total time to score all 19 schedules of one instance; candidate generation is excluded.

Decision metrics are computed within each instance before aggregation, so the effective sample size is 100 rather than 1,900 schedules. Kendall’s τ_b and Inv. handle ties as defined in Section III-B; selection regret uses the candidate order fixed before test evaluation. The executor rejects 26 of the 9,500 schedules (0.27%); decision metrics use the common executor-routable candidate set, whereas each time observation totals all 19 evaluator calls for one instance before the mean is computed.

C. Terminal Return and Congestion Observability

Table V gives the single-seed best-of-eight factorial, with surrogate configurations in the rightmost block. Both executor and surrogate returns substantially improve on the idealized-return controls; the comparison of greatest practical interest is therefore Surr–Cong versus Exec–Cong.

Under O_{cong} , Surr–Cong is better than Exec–Cong from 40 through 100 jobs, whereas the same pattern is not observed under O_{core} . Best-of-eight strengthens the result: Surr–Cong is better at all five workloads by 2.0–15.5 ticks. Thus, a higher-fidelity executor return does not automatically yield the better policy under surrogate-based state transitions.

D. Comparison with Rules and Genetic Search

Table VI compares Surr–Cong with baselines fixed without using the test set and reports inference time. Greedy performance is mixed, whereas best-of-eight reduces rule makespan by 4.1–9.1% at every workload. It is 3.1% worse than GA at $n = 20$, but 3.6–7.9% better from $n = 40$ through 100.

Best-of-eight produces no routing failures in the reported Surr–Cong results; greedy decoding fails in 7 of 1,500 seed-instance evaluations. As specified in the protocol, these failures are not treated as completed makespans.

E. Computation Time

We separate evaluator cost from end-to-end scheduling cost. Table IV reports the mean elapsed time per test instance for scoring its 19 candidates; it excludes candidate generation.

The scheduling-time columns of Table VI report serial, single-process results on an NVIDIA GeForce RTX 5080. The common final reporting call to Φ is excluded, while executor calls used for best-of- K selection are included.

Training required approximately 55–74 hours per run under shared GPU load. Because contention differed and validation still invokes Φ , these values document resource use but do not support a controlled whole-training speedup claim.

VI. CONCLUSION

This paper formulates offline capacitated multi-robot pickup and delivery with explicit station contention and evaluation by a fixed collision-aware executor. It distinguishes the value error of an idealized transport model from its more consequential decision error—changes in the ordering of complete schedules—and trains a constructive policy from executor-evaluated terminal cost while using a calibrated surrogate for intermediate states. The controlled evaluation separates improvements due to congestion observability from those due to the training objective. The idealized model substantially misranks schedules once the candidate set includes policies optimized against it: at the 60-job training workload, 38.4% of comparable pairs are inverted and trusting its selected schedule loses 30.6% under execution. Executor grounding is the dominant causal factor, while projected congestion adds a smaller effect that grows

TABLE IV
EVALUATOR FIDELITY AND SCORING TIME.

n	Executor Φ	Idealized $\tilde{C}$				Surrogate $\hat{\Psi}$			
	t (ms)	τ_b	Inv. (%)	Reg. (%)	t (ms)	τ_b	Inv. (%)	Reg. (%)	t (ms)
20	664.66	0.26 ± 0.04	36.25	16.08	1.18	0.69 ± 0.03	14.91	3.62	2.79
40	1279.83	0.13 ± 0.03	43.22	38.62	2.03	0.73 ± 0.02	13.04	2.69	5.17
60	1915.73	0.23 ± 0.03	38.40	30.63	2.84	0.75 ± 0.02	12.28	3.19	7.58
80	2636.45	0.30 ± 0.03	34.96	27.51	3.65	0.77 ± 0.02	11.06	2.65	10.01
100	3243.23	0.37 ± 0.02	31.39	9.41	4.31	0.81 ± 0.01	9.50	1.28	12.22

TABLE V
BEST-OF-EIGHT FACTORIAL RESULTS.

n	Ideal return		Executor return		Surrogate return	
	Core	Cong	Core	Cong	Core	Cong
20	192.70 ± 3.60	222.72 ± 4.20	160.18 ± 2.96	161.14 ± 2.93	160.16	2.65
40	378.69 ± 6.05	325.48 ± 4.39	246.58 ± 2.57	242.15 ± 3.08	253.51 ± 3.07	239.23
60	603.03 ± 7.03	419.62 ± 4.74	338.48 ± 2.11	326.07 ± 3.05	351.88 ± 2.52	319.89
80	799.95 ± 6.34	524.35 ± 4.46	436.05 ± 2.61	416.56 ± 3.16	450.99 ± 3.21	403.43
100	968.25 ± 8.27	630.15 ± 5.51	534.46 ± 2.49	504.30 ± 3.47	550.14 ± 3.41	488.50

TABLE VI
EXECUTED MAKESPAN AND SCHEDULING TIME.

n	Rule		GA		Surr-G		Surr-8	
	M	t	M	t	M	t	M	t
20	166.22 ± 2.82	0.03	153.95	2.84	15.73	168.65 ± 3.16	0.10	158.78 ± 2.93
40	258.38 ± 3.78	0.10	246.34 ± 3.00	37.21	249.39 ± 3.83	0.35	237.54	3.14
60	349.61 ± 3.57	0.23	338.68 ± 2.98	65.85	334.75 ± 3.34	0.77	317.95	2.65
80	426.24 ± 7.22	0.57	435.85 ± 4.02	101.86	422.39 ± 3.94	1.36	401.38	3.40
100	505.98 ± 8.02	0.89	525.85 ± 4.07	144.93	513.23 ± 4.46	2.15	485.11	3.11

with workload. The calibrated surrogate recovers much of the executor’s ranking at roughly 1/250 of its scoring time. On the fixed layout, the executor-scored eight-rollout policy improves on strong rule and genetic baselines, although single-rollout generalization weakens away from its 60-job training distribution. Extending the study to other layouts, fleets, and complete multi-agent routers is the principal direction for future work.

REFERENCES

- [1] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and X. Chi, “Learning to dispatch for job shop scheduling via deep reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] W. Song, X. Chen, Q. Li, and Z. Cao, “Flexible job-shop scheduling via graph neural network and deep reinforcement learning,” *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1600–1610, 2023.
- [3] S. Moon, S. Lee, and K.-J. Park, “Learning-enabled flexible job-shop scheduling for scalable smart manufacturing,” *Journal of Manufacturing Systems*, vol. 77, pp. 356–367, 2024.
- [4] M. Yuan, Y. Ye, L. Zheng, Z. Zhang, F. Pei, and Y. Han, “Research on the flexible job-shop scheduling problem considering AGV transport time based on proximal policy optimization with a graph isomorphism network,” *Flexible Services and Manufacturing Journal*, 2026.
- [5] Y. Shen, X. Zhang, and T. Jin, “Transformer-based multi-agent reinforcement learning for flexible job shop scheduling with AGVs,” *Applied Soft Computing*, p. 114899, 2026.
- [6] Z. Chen, J. Alonso-Mora, X. Bai, D. D. Harabor, and P. J. Stuckey, “Integrated task assignment and path planning for capacitated multi-agent pickup and delivery,” *arXiv preprint arXiv:2110.14891*, 2021.
- [7] S. Li, L. Fan, and S. Jia, “A hierarchical solution framework for dynamic and conflict-free AGV scheduling in an automated container terminal,” *Transportation Research Part C: Emerging Technologies*, vol. 165, p. 104724, 2024.

- [8] Y.-J. Wang, R. Hu, B. Qian, K. Li, W.-B. Zhang, and J.-B. Yang, “Collaborative deep reinforcement learning algorithm for solving multi-AGV dynamic scheduling problem,” *Expert Systems with Applications*, vol. 319, p. 131941, 2026.

- [9] L. Yang, Z. Yang, L. Bi, and X. Jiao, "Dynamic flexible job shop co-scheduling optimization based on graph neural network and deep reinforcement learning," *Operations Research Perspectives*, vol. 16, 2026.
- [10] Ü. Bilge and G. Ulusoy, "A time window approach to simultaneous scheduling of machines and material handling system in an FMS," *Operations Research*, vol. 43, no. 6, pp. 1058–1070, 1995.
- [11] G. Ulusoy, F. Sivrikaya-Şerifoğlu, and Ü. Bilge, "A genetic algorithm approach to the simultaneous scheduling of machines and automated guided vehicles," *Computers & Operations Research*, vol. 24, no. 4, pp. 335–351, 1997.
- [12] T. Miyamoto and K. Inoue, "Local and random searches for dispatch and conflict-free routing problem of capacitated AGV systems," *Computers & Industrial Engineering*, vol. 91, pp. 1–9, 2016.
- [13] S. Ji, D. Luan, Z. Chen, and D. Guo, "Integrated scheduling in automated container terminals considering AGV conflict-free routing," *Transportation Letters*, vol. 13, no. 7, pp. 501–513, 2021.
- [14] K. Nishida and T. Nishi, "Dynamic optimization of conflict-free routing of automated guided vehicles for just-in-time delivery," *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 3, pp. 2099–2114, 2023.
- [15] K. Li, T. Liu, P. N. Ram Kumar, and X. Han, "A reinforcement learning-based hyper-heuristic for AGV task assignment and route planning in parts-to-picker warehouses," *Transportation Research Part E: Logistics and Transportation Review*, vol. 185, p. 103518, 2024.
- [16] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig *et al.*, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proc. Symposium on Combinatorial Search (SoCS)*, 2019.
- [17] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding for online pickup and delivery tasks," in *Proc. International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2017.
- [18] H. Zheng, Y. Ma, B. Araki, J. Chen, and C. Wu, "Learning-guided prioritized planning for lifelong multi-agent path finding in warehouse automation," *Journal of Artificial Intelligence Research*, vol. 85, p. Article 28, 2026.
- [19] T. Hirayama, K. Yoshida, H. Sakaji, and I. Noda, "Deliberate assignment deferral for multi-agent pickup and delivery with deadlines," *Systems*, vol. 14, no. 5, p. 494, 2026.
- [20] C. Zang, I. Barz, A. Mannucci, P. Schillinger, F. Lier, and W. Hönig, "Lifelong multi-subsystem pickup and delivery with buffer-limited handover stations," *arXiv preprint arXiv:2607.17724*, 2026.
- [21] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [23] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *International Conference on Learning Representations (ICLR)*, 2019.